

Andrew needs to put it into mission scene, Itv scene,

Note: Please don't reimport the XR Hands package or mess with the settings

There also cannot be two EventSystems. I already created one, and so you should not need to create one.



NASA SUITS 2026: XR Hand Tracking & Interaction Guide

This document explains our new **XR Origin** setup in the **Main** scene and how to use the hand-tracking features to build UI and interactable objects for the Magic Leap 2.

Part 1: Anatomy of the New XR Rig

We have upgraded from the old **ML Rig** to a standard Unity **XR Origin** powered by the XR Interaction Toolkit and OpenXR. Here is what each part of the rig actually does:



The Core System

- **XR Origin:** The absolute center of the player's physical tracking space. It natively manages the alignment between the real world and the Unity world, handling 6 Degrees of Freedom (6DoF) headset movement and OpenXR hand-tracking skeletons without needing an external manager.
- **Main Camera:** Tracks the position and rotation of the Magic Leap 2 headset. Our **UIAnchor** is attached here so the HUD follows the user's gaze.

The hand system includes interactors to allow for different manipulations; the primary interactors include Direct Interactor (grabbing/touching), Ray Interactor (distant pointing, also distance grabbing with ray), and Poke Interactor (pushing UI/buttons). These allow the hands to potentially use these functionalities, but only on objects that contain the related components and necessary structures. The components are similarly-named (e.g. XR Grab Interactable).



Hands vs. Controllers

Under the Camera Offset, you will notice both "Hands" and "Controllers" objects.

- **Left/Right Controllers:** These handle the standard tracking data (where the hands or physical Magic Leap clicker are in space) and process physical button presses.
 - **Left/Right Hands:** These contain the visual skeletal meshes and the specific "Interactors" (the scripts that allow you to actually touch and grab things).
-

Part 2: The Interactors (How we touch the world)

The new hands come equipped with three specific ways to interact with the digital world.

1. XR Ray Interactor (The Laser Pointer)

- **What it is:** A curved line that shoots out from the palm/fingers.
- **What it's for:** Interacting with menus or objects from far away. If the user pinches their fingers while pointing at a UI button, it acts like a mouse click.

2. XR Poke Interactor (The Finger Press)

- **What it is:** A small sphere collider at the tip of the index finger.
- **What it's for:** Pushing physical-style 3D buttons or directly poking standard Unity UI Canvases as if they were touchscreens.

3. XR Direct Interactor (The Grabber)

- **What it is:** A trigger zone surrounding the hand itself.
- **What it's for:** Reaching out and grabbing 3D objects (like virtual tools, rock samples, or 3D map waypoints) when your hand is physically intersecting the object.

Tutorials can be found below.

Tutorials

Detailed Tutorial on Creating Grabbable UI Objects

Step 1: Create the Physical Menu Board

We need to create the container that will hold both your UI and your physics colliders.

1. Right-click in your Hierarchy and create an **Empty GameObject**. Name it something relevant.
2. Add a **Box Collider** to it. Adjust the size of the box so it matches the rough size of what your screen will be (e.g., **X: 0.5, Y: 0.3, Z: 0.05**).
3. Add an **XR Grab Interactable** component to this object.
4. Unity will automatically add a **Rigidbody**. Click on the Rigidbody and check the box that says **Is Kinematic** (this ensures your menu doesn't fall out of the sky and hit the virtual floor).

Step 2: Build the UI Canvas

Now we add the actual visual screen as a child of that panel.

1. Right-click your menu and select **UI > Canvas**.
2. Change the Canvas Render Mode to **World Space**.
3. Scale the Canvas down significantly (e.g., set the Scale to **X: 0.001, Y: 0.001, Z: 0.001**) and adjust its Width and Height so it fits nicely inside your Box Collider.
4. Drag your **Main Camera** from the XR Origin into the Canvas's **Event Camera** slot.
5. **Crucial Step:** Remove the default **Graphic Raycaster** component from the Canvas and add the **Tracked Device Graphic Raycaster**. (This allows your XR lasers to click the buttons on the screen).
6. Add your placeholder UI elements (Text boxes for Vitals, Oxygen, Map, etc.) as children of this Canvas.

Common question I get asked: How do I implement button pressing for my UI that's directly on the screen? It's not a UI object like your menus. Answered by Gemini:

Question: Can someone create a UI that's on the screen (Screen-Space Overlay) and have it pressable, rather than a 3D object UI (World Space)?

The Short Answer: In modern XR development (especially with hand tracking), **No, you practically cannot.** You must use World Space UI.

Here is the technical reason why:

1. The Interaction Paradigm

Unity's default Screen-Space Overlay UI is designed for 2D mice and touchscreens. It takes the UI and physically renders it flat over the camera's final image on the lens.

XR Interaction requires **spatial depth**.

- **Poke Interactor:** Your finger needs to physically pass through the plane of a button in 3D space to trigger it.
- **Ray Interactor:** A laser pointer shoots a ray out from your hand into the scene looking for 3D physics colliders to hit.

If a Canvas is set to Screen-Space Overlay, it has no position, size, or collider in the virtual world. The XR hand ray will look straight "through" it at the wall behind it, and your finger cannot poke it because the UI only exists as a flat image on the glasses, not as an object in front of your face.

2. Physiological/Safety Reasons

Even if you *simulated* interaction (by mapping hand position against screen coordinates):

- **No Accommodation-Vergence:** Your eyes would be trying to focus on a digital screen only an inch away (the ML2 lenses), while simultaneously trying to look at the physical room 10 feet away. This is the fastest way to cause severe headaches and motion sickness in XR.
- **Blocking View:** An astronaut needs an unobstructed view. Rigidly face-locking menus that block the physical environment is unsafe.

The Correct Approach

For modern XR, you must use **World Space Canvases**, combined with techniques like "Body-Locking" or "Lazy Follow" to keep the menus accessible but spatially valid.

Part 2: Tutorial - Creating Interactable XR Hand Buttons

This is a quick guide on how to create a standard Unity UI Button (TextMeshPro) and ensure it works perfectly with your new XRI hand rig.

Phase 1: Setup the World Space Canvas

XR Hand interaction *only* works with World Space Canvases.

1. Create a Canvas (**GameObject > UI > Canvas**). Name it **SpatialCanvas**.
 2. Select the Canvas, find the **Canvas component** in the inspector, and change the **Render Mode** to **World Space**.
 3. Drag the main **XR Camera** (the one childed to the XR Origin) into the **Event Camera** slot.
 4. Scale the Canvas down! (Set **Scale X, Y, Z** to **0.001, 0.001, 0.001**). This makes 1 pixel equal 1 millimeter.
 5. In the RectTransform, set **Width** and **Height** (e.g., **800** by **600**) and position the Canvas somewhere the astronaut can see it.
-

Phase 2: Add the Raycaster (THE MOST IMPORTANT STEP)

By default, Canvases have a **Graphic Raycaster**. This will not work with hands.

1. Scroll down on the Canvas inspector to find the **Graphic Raycaster** component. **Remove it**.
 2. Add the **Tracked Device Graphic Raycaster** component instead. (*This specific raycaster knows how to listen to the pointers on the XR hands*).
-

Phase 4: Create the Button

1. Right-click your Canvas in the hierarchy and select **UI > Button - TextMeshPro**. (Scale and position it as desired on your canvas).

(Note: We do not need to add Box Colliders to the buttons themselves. The Tracked Device Graphic Raycaster handles the hits on the visual Image components).

Phase 5: Hook up Functionality (C#)

Now you need to tell the button what to do when an astronaut clicks/pokes it.

1. Create a placeholder C# script (e.g., `NASA_UISystem`).
2. Add a simple test method:

C#

```
None
using UnityEngine;

public class NASA_UISystem : MonoBehaviour
{
    // Make sure it is PUBLIC so the button can see it
    public void VitalsButton_OnClick()
    {
        Debug.Log("<color=green> Astronaut Vitals Toggled!</color>");
    }
}
```

3. Select your **Button** in the hierarchy.
4. Scroll down in the inspector to the **Button component** and find the **On Click ()** event list.
5. Click the **+** (plus sign) to add an event.
6. Drag the object holding your `NASA_UISystem` script into the object slot.
7. Use the dropdown function list to select `NASA_UISystem > VitalsButton_OnClick`.

Phase 6: How to Interact in the Headset

Now build and run. The astronaut has two ways to click this new button:

1. **The Laser Poke (Far Interaction):** Point your hand laser at the button. Pinch your thumb and index finger together. This will trigger the button's `OnClick()` event from a distance.
2. **The Finger Poke (Direct Interaction):** If the canvas is physically close, you can simply reach out and "poke" the visual button with your index finger, just like a touchscreen on an iPad. The XRI rig will detect your finger collider passing through the Canvas plane and trigger the click.

And then some information about the difference between the current hand tracking model vs the one that was in maps:

Analyzing Your Teammate's Setup

Here is exactly what is happening in their map_test scene based on the screenshot:

They are using a basic Controller Rig, not a Hand Rig: Their XR Origin only has Left Controller and Right Controller. In Unity, this is the default setup meant for physical hardware controllers (like Quest controllers or the Magic Leap 2 physical wand).

Why they can pinch: The Magic Leap OS maps simple hand gestures (like pinching) to standard controller buttons (like pulling a trigger). Their PinchPingSpawner script listens for this raw input.

Where are their hands? Since there is no Hand Visualizer or hand mesh in their scene, they might be relying on the Magic Leap OS's native system overlay to see their hands, or they might just be seeing the orbs spawn in thin air without a 3D hand model attached.

Verifying Why Your Rig is Superior for UI

With this factual layout, my previous claims about your rig being the necessary choice for your goals remain completely true. Here is the verified technical breakdown of why:

Claim 1: Their rig cannot push a button physically. * Proof: To push a Canvas button physically in 3D space, Unity requires a Poke Interactor component. Their hierarchy does not have one. If you put a button in front of their Left Controller, nothing will happen when their finger touches it because there is no sensor tracking the fingertip. Your rig has Left Hand > Poke Interactor.

Claim 2: Their rig cannot pinch-to-drag UI frames out-of-the-box.

Proof: To drag objects, XRI requires an XR Grab Interactor (or Direct Interactor) attached to the hands. Their Left Controller is just an empty transform container or a basic raycaster at best. Your rig has Direct Interactor built into the hand structure.

Claim 3: Your rig is officially built for OpenXR Hands.

Proof: Your hierarchy (from the earlier screenshots) explicitly maps the Left Hand and Right Hand structures, which is what the XR Hand Skeleton Driver needs to process all 26 joints. Their rig is completely missing the hand data structure required to know where an index finger or thumb actually is in space.

The Good News About the Merge

Because they don't have a custom Hand Visualizer or a complex visual setup that needs to be preserved, merging your work is actually much easier than I previously thought.

You don't need to try and copy your components onto their empty controllers. You can simply do this:

Bring your entire XR Origin (XR Rig) into the map_test scene.

Delete or disable their barebones XR Origin.

Attach their PinchPingSpawner script to an empty GameObject (like a "GameManager") so they keep their orb functionality.

Your rig will drop in, instantly give you the Poke and Ray interactions for the Canvas 1 UI, and their script will continue to work perfectly alongside it.